

DevOps & AWS Complete Learning Guide

1 DevOps & AWS Learning Hub

[1.1 Download PDFs](#)

[1.2 Suggested study pace](#)

[1.3 Page convention](#)

2 DevOps Curriculum: Tools & Concepts (Beginner → Advanced)

[2.1 Level 0 — Prerequisites \(before DevOps\)](#)

[2.1.1 Concepts](#)

[2.1.2 Tools & Skills](#)

[2.2 Level 1 — DevOps Foundations](#)

[2.2.1 Concepts](#)

[2.2.2 Tools](#)

[2.3 Level 2 — Infrastructure & Automation](#)

[2.3.1 Concepts](#)

[2.3.2 Tools](#)

[2.4 Level 3 — Containers & Orchestration](#)

[2.4.1 Concepts](#)

[2.4.2 Tools](#)

[2.5 Level 4 — CI/CD Pipelines](#)

[2.5.1 Concepts](#)

[2.5.2 Tools](#)

[2.6 Level 5 — Cloud Platforms \(cross-cutting\)](#)

[2.6.1 Concepts](#)

[2.6.2 Tools \(by provider\)](#)

[2.7 Level 6 — Observability & Reliability](#)

[2.7.1 Concepts](#)

[2.7.2 Tools](#)

[2.8 Level 7 — Security \(DevSecOps\)](#)

[2.8.1 Concepts](#)

[2.8.2 Tools](#)

[2.9 Level 8 — Data, Messaging & Databases \(Ops view\)](#)

[2.9.1 Concepts](#)

[2.9.2 Tools](#)

[2.10 Level 9 — Platform Engineering & Internal Developer Platforms \(adv\)](#)

[2.10.1 Concepts](#)

[2.10.2 Tools](#)

[2.11 Level 10 — Advanced & Specialized \(adv\)](#)

[2.11.1 Concepts](#)

[2.11.2 Tools](#)

[2.12 Certifications \(optional milestones\)](#)

[2.13 Learning project ideas \(apply everything\)](#)

3 AWS Cloud Curriculum: Services & Tools (Beginner → Advanced)

[3.1 1. Account, Identity & Governance ★](#)

[3.1.1 Concepts](#)

[3.2 2. Networking & Content Delivery ★](#)

[3.2.1 Tools](#)

[3.3 3. Compute ★](#)

[3.3.1 Tools](#)

[3.4 4. Storage ★](#)

[3.4.1 Concepts](#)

[3.5 5. Databases ★](#)

[3.6 6. Application Integration & Messaging ★](#)

[3.7 7. Developer Tools & CI/CD ★](#)

[3.8 8. Observability & Management ★](#)

[3.9 9. Security, Identity & Compliance ★](#)

[3.10 10. Migration & Transfer ♦](#)

[3.11 11. Analytics & Big Data ♦](#)

[3.12 12. Machine Learning & AI ◇](#)

[3.13 13. IoT, Media & Specialized ◇](#)

[3.14 14. Cost & Billing ★](#)

[3.14.1 Concepts](#)

[3.15 15. Well-Architected Framework \(conceptual ★\)](#)

[3.16 Certification path \(recommended order\)](#)

[3.17 Hands-on lab progression](#)

[4 DevOps Handbook — Master Table of Contents](#)

[4.1 How to read this handbook](#)

[4.2 Part I — Introduction & Mindset \(Pages 1–60\)](#)

[4.3 Part II — Linux & Shell Mastery \(Pages 61–140\)](#)

[4.4 Part III — Git & Collaboration \(Pages 141–200\)](#)

[4.5 Part IV — Networking for DevOps \(Pages 201–280\)](#)

[4.6 Part V — Scripting & Programming \(Pages 281–360\)](#)

[4.7 Part VI — Infrastructure as Code \(Pages 361–480\)](#)

[4.8 Part VII — Containers \(Pages 481–580\)](#)

[4.9 Part VIII — Kubernetes \(Pages 581–720\)](#)

[4.10 Part IX — CI/CD \(Pages 721–840\)](#)

[4.11 Part X — Observability & SRE \(Pages 841–960\)](#)

[4.12 Part XI — Security \(DevSecOps\) \(Pages 961–1080\)](#)

[4.13 Part XII — Advanced Topics \(Pages 1081–1200\)](#)

[4.14 Expansion status](#)

[5 Chapter 1: What Is DevOps? History and Culture](#)

[5.1 1.1 The problem DevOps solves](#)

[5.2 1.2 A working definition](#)

[5.3 1.3 Brief history](#)

[5.4 1.4 Culture before tools](#)

[5.5 1.5 DevOps practices \(overview\)](#)

[5.6 1.6 Organizational models](#)

[5.6.1 Functional silos \(anti-pattern for DevOps\)](#)

[5.6.2 Cross-functional product teams \(recommended\)](#)

[5.6.3 DevOps enablement / platform team](#)

[5.7 1.7 What DevOps is NOT](#)

[5.8 1.8 Your learning path in this handbook](#)

[5.9 1.9 Chapter summary](#)

[5.10 Lab 1.1 — Reflect on your current workflow](#)

[5.11 Review questions](#)

[6 Chapter 2: CALMS, Three Ways, and Team Topologies](#)

[6.1 2.1 The CALMS framework](#)

[6.1.1 Culture \(C\)](#)

[6.1.2 Automation \(A\)](#)

[6.1.3 Lean \(L\)](#)

[6.1.4 Measurement \(M\)](#)

[6.1.5 Sharing \(S\)](#)

[6.2 2.2 The Three Ways \(Gene Kim\)](#)

[6.2.1 First Way: Flow \(left to right\)](#)

[6.2.2 Second Way: Feedback \(right to left\)](#)

[6.2.3 Third Way: Continuous learning and experimentation](#)

[6.3 2.3 Team Topologies \(Skelton & Pais\)](#)

[6.3.1 Interaction modes](#)

[6.4 2.4 Conway's Law and reverse Conway maneuver](#)

[6.5 2.5 Maturity model \(informal\)](#)

[6.6 2.6 Chapter summary](#)

[6.7 Lab 2.1 — CALMS assessment](#)

[6.8 Review questions](#)

[7 Chapter 5: Linux Fundamentals — Filesystem, Users, Permissions](#)

[7.1 5.1 Why Linux dominates DevOps](#)

[7.2 5.2 Filesystem hierarchy \(FHS\)](#)

[7.3 5.3 Essential navigation commands](#)

[7.4 5.4 File operations](#)

[7.4.1 Finding files](#)

[7.5 5.5 Users, groups, and permissions](#)

[7.5.1 Users and groups](#)

[7.5.2 Permission model](#)

[7.5.3 Special permissions](#)

[7.6 5.6 Package management \(Debian/Ubuntu\)](#)

[7.7 5.7 Text processing \(daily DevOps tools\)](#)

[7.8 5.8 Environment and shell](#)

[7.9 5.9 SSH — remote access](#)

[7.10 5.10 Chapter summary](#)

[7.11 Lab 5.1 — Linux sandbox](#)

[7.12 Review questions](#)

[8 AWS Handbook — Master Table of Contents](#)

[8.1 Part I — Cloud Foundations \(Pages 1–80\)](#)

[8.2 Part II — Identity & Access Management \(Pages 81–160\)](#)

[8.3 Part III — Networking \(Pages 161–280\)](#)

[8.4 Part IV — Compute \(Pages 281–400\)](#)

[8.5 Part V — Storage \(Pages 401–480\)](#)

[8.6 Part VI — Databases \(Pages 481–560\)](#)

[8.7 Part VII — Application Integration \(Pages 561–620\)](#)

[8.8 Part VIII — DevOps on AWS \(Pages 621–700\)](#)

[8.9 Part IX — Observability & Operations \(Pages 701–760\)](#)

[8.10 Part X — Security & Compliance \(Pages 761–800\)](#)

[8.11 Expansion status](#)

[9 Chapter 1: Introduction to AWS & the Shared Responsibility Model](#)

[9.1 1.1 What is Amazon Web Services \(AWS\)?](#)

[9.1.1 Why AWS matters for DevOps](#)

[9.2 1.2 Cloud service models](#)

[9.3 1.3 The Shared Responsibility Model](#)

[9.3.1 AWS is responsible for **Security OF the Cloud**](#)

[9.3.2 You are responsible for **Security IN the Cloud**](#)

[9.3.3 Example: RDS vs EC2 MySQL](#)

[9.4 1.4 Core AWS concepts](#)

[9.5 1.5 AWS console, CLI, and IaC](#)

[9.6 1.6 Pricing fundamentals](#)

[9.7 1.7 Well-Architected Framework \(preview\)](#)

[9.8 1.8 Chapter summary](#)

[9.9 Lab 1.1 — Create your AWS account](#)

[9.10 Review questions](#)

[10 Chapter 2: AWS Global Infrastructure — Regions, AZs, and Edge](#)

[10.1 2.1 Global infrastructure overview](#)

[10.2 2.2 Regions](#)

[10.2.1 Choosing a region](#)

[10.2.2 Regional vs global services](#)

[10.3 2.3 Availability Zones \(AZs\)](#)

[10.3.1 Multi-AZ patterns](#)

[10.4 2.4 Local Zones, Wavelength, and Outposts](#)

[10.5 2.5 Edge locations and CloudFront](#)

[10.6 2.6 Fault isolation design principles](#)

[10.6.1 S3 durability vs availability](#)

[10.7 2.7 Service quotas \(limits\)](#)

[10.8 2.8 Chapter summary](#)

[10.9 Lab 2.1 — Explore regions in CLI](#)

[10.10 Review questions](#)

[11 Chapter 5: IAM Users, Groups, Roles, and Policies](#)

[11.1 5.1 What is IAM?](#)

[11.2 5.2 IAM identity types](#)

[11.3 5.3 Policies — the core of permissions](#)

[11.3.1 Policy structure](#)

[11.3.2 Policy types](#)

[11.4 5.4 Users and groups](#)

[11.4.1 Create a developer group \(example workflow\)](#)

[11.5 5.5 Roles and trust policies](#)

[11.5.1 EC2 instance role example](#)

[11.5.2 Assume role \(cross-account or CLI\)](#)

[11.6 5.6 AWS managed vs customer managed policies](#)

[11.7 5.7 Least privilege in practice](#)

[11.7.1 Tag-based access \(ABAC\)](#)

[11.8 5.8 Common IAM actions for DevOps](#)

[11.9 5.9 IAM Access Analyzer & CloudTrail](#)

[11.10 5.10 Chapter summary](#)

[11.11 Lab 5.1 — IAM hands-on](#)

[11.12 Review questions](#)

1 DevOps & AWS Learning Hub

A structured path from zero to advanced, plus two handbook series:

Resource	Target size	Status
DevOps Curriculum	Reference list	Complete
AWS Curriculum	Reference list	Complete
DevOps Handbook	~1,200 pages	TOC + Part I-II written; Parts III-XII outlined
AWS Handbook	~800 pages	TOC + Part I-II written; Parts III-X outlined

1.1 Download PDFs

Pre-built PDFs are in the [pdf/](#) folder:

PDF	Description
devops-handbook.pdf	DevOps handbook (TOC + written chapters)
aws-handbook.pdf	AWS handbook (TOC + written chapters)
devops-curriculum.pdf	DevOps tools & concepts reference
aws-curriculum.pdf	AWS services reference
devops-aws-complete-guide.pdf	All content in one PDF

Rebuild PDFs after editing markdown:

```
./scripts/build-pdfs.sh
```

Requires `pandoc` and `wkhtmltopdf`.

1. **Start with the curricula** — they list every tool, concept, and AWS service in learning order.
2. **Follow the handbook TOCs** — each chapter has a page budget; work through sequentially.
3. **Expand on demand** — ask an AI assistant or instructor to flesh out any chapter number from the TOC.

1.2 Suggested study pace

Phase	Duration (self-paced)	Focus
Foundations	4-8 weeks	Linux, Git, networking, scripting
Core DevOps	8-12 weeks	CI/CD, containers, IaC, cloud basics
Intermediate	8-12 weeks	Kubernetes, observability, security
		Cloud platform

Advanced	12+ weeks	SRE, platform engineering, multi-cloud, FinOps
----------	-----------	--

1.3 Page convention

Each handbook “page” $\approx$ 400 words of instructional content + diagrams/commands. Labs and exercises add extra pages in the TOC estimates.

2 DevOps Curriculum: Tools & Concepts (Beginner → Advanced)

Complete reference of what to learn, in recommended order. Items marked **(adv)** are advanced topics.

2.1 Level 0 — Prerequisites (before DevOps)

2.1.1 Concepts

- Software development lifecycle (SDLC)
- Agile, Scrum, Kanban
- Waterfall vs iterative delivery
- Client-server architecture
- APIs (REST, GraphQL basics)
- Databases (SQL vs NoSQL overview)
- Environment tiers: dev, staging, production

2.1.2 Tools & Skills

Category	Tools / Skills
Operating systems	Linux (Ubuntu, RHEL), Windows Server basics
Shell	Bash, Zsh, PowerShell basics
Text editing	Vim, Nano, VS Code
Networking	TCP/IP, DNS, HTTP/HTTPS, TLS, ports, firewalls
Version control	Git (clone, branch, merge, rebase, cherry-pick)
Scripting	Python, Bash, Go (pick one primary)
Documentation	Markdown, diagrams (draw.io, Mermaid)

2.2 Level 1 — DevOps Foundations

2.2.1 Concepts

- What is DevOps? Culture, CALMS (Culture, Automation, Lean, Measurement, Sharing)
- DevOps vs SysAdmin vs SRE vs Platform Engineering
- Continuous Integration (CI) vs Continuous Delivery (CD) vs Continuous Deployment
- Build artifacts, immutable infrastructure
- Configuration vs secrets
- The Twelve-Factor App
- DevSecOps introduction
- GitOps introduction

2.2.2 Tools

Category	Tools
Git hosting	GitHub, GitLab, Bitbucket, Azure DevOps Repos
SSH & keys	OpenSSH, ssh-agent, key pairs

Package managers	apt, yum/dnf, Homebrew
Build tools	Make, Maven, Gradle, npm/yarn/pnpm
Virtualization	VirtualBox, VMware, Hyper-V
Cloud CLI basics	AWS CLI, gcloud, az

2.3 Level 2 — Infrastructure & Automation

2.3.1 Concepts

- Infrastructure as Code (IaC) principles
- Idempotency, declarative vs imperative
- Server provisioning & bootstrapping
- Configuration management
- Immutable vs mutable infrastructure
- Golden images / AMIs
- Inventory & state management

2.3.2 Tools

Category	Tools
IaC — Terraform	Terraform, OpenTofu, Terragrunt
IaC — Cloud-native	AWS CloudFormation, Azure Bicep/ARM, Google Deployment Manager, Pulumi, CDK (AWS/Azure/GCP)
Configuration mgmt	Ansible, Chef, Puppet, SaltStack
Image building	Packer
Secrets (intro)	HashiCorp Vault, AWS Secrets Manager, SOPS
Cloud VMs	EC2, Azure VMs, GCE

2.4 Level 3 — Containers & Orchestration

2.4.1 Concepts

- Containers vs VMs
- Images, layers, registries
- Dockerfile best practices
- Container networking & storage
- Orchestration primitives (Pod, Service, Deployment)
- Service mesh concepts
- Sidecar pattern
- Helm charts & package management

2.4.2 Tools

Category	Tools
Runtime	Docker, containerd, CRI-O, Podman
Compose / local	Docker Compose, Dev Containers

Registries	Docker Hub, ECR, GCR, ACR, Harbor, Artifactory
Orchestration	Kubernetes (K8s), Amazon ECS, Amazon EKS, GKE, AKS
K8s tooling	kubectl, k9s, Lens, Helm, Kustomize
Service mesh	Istio, Linkerd, Consul Connect
Local K8s	minikube, kind, k3s, Docker Desktop K8s

2.5 Level 4 — CI/CD Pipelines

2.5.1 Concepts

- Pipeline stages: source → build → test → scan → deploy
- Branching strategies (GitFlow, trunk-based)
- Artifact versioning & promotion
- Blue/green, canary, rolling deployments
- Feature flags
- Pipeline as Code
- DORA metrics (deployment frequency, lead time, MTTR, change failure rate)

2.5.2 Tools

Category	Tools
CI/CD platforms	Jenkins, GitHub Actions, GitLab CI, CircleCI, Travis CI, Azure Pipelines, AWS CodePipeline, Tekton, Argo CD (GitOps)
Build agents	Jenkins agents, GitHub-hosted/self-hosted runners
Artifact repos	Nexus, Artifactory, GitHub Packages
GitOps	Argo CD, Flux, Fleet
Deployment	Spinnaker, Harness, Octopus Deploy

2.6 Level 5 — Cloud Platforms (cross-cutting)

2.6.1 Concepts

- Shared responsibility model
- Regions, AZs, edge locations
- IAM (users, roles, policies, least privilege)
- VPC networking (subnets, routing, NAT, peering)
- Load balancing (L4 vs L7)
- Object storage vs block vs file
- Managed vs self-managed services
- Well-Architected Framework pillars
- Cost optimization & tagging
- Multi-account / landing zone strategy

2.6.2 Tools (by provider)

Provider	Core services to learn
----------	------------------------

AWS	See curriculum-aws.md
Azure	Resource Manager, AAD, VNet, AKS, App Service
GCP	IAM, VPC, GKE, Cloud Run, Cloud Build

2.7 Level 6 — Observability & Reliability

2.7.1 Concepts

- Monitoring vs observability (metrics, logs, traces)
- SLI, SLO, SLA, error budgets
- Alerting philosophy (symptom vs cause)
- On-call, incident management, postmortems
- Chaos engineering
- Capacity planning
- Distributed tracing

2.7.2 Tools

Category	Tools
Metrics	Prometheus, Grafana, Datadog, New Relic, CloudWatch, Azure Monitor
Logging	ELK/EFK (Elasticsearch, Logstash/Fluentd, Kibana), Loki, Splunk, CloudWatch Logs
Tracing	Jaeger, Zipkin, OpenTelemetry, AWS X-Ray, Honeycomb
APM	Datadog APM, Dynatrace, AppDynamics
Incident mgmt	PagerDuty, Opsgenie, incident.io
Status	Statuspage
Chaos	Chaos Monkey, LitmusChaos, Gremlin

2.8 Level 7 — Security (DevSecOps)

2.8.1 Concepts

- Shift-left security
- OWASP Top 10
- Supply chain security (SBOM, signing)
- Zero trust networking
- CIS benchmarks
- Compliance frameworks (SOC 2, PCI-DSS, HIPAA overview)
- Threat modeling
- Principle of least privilege

2.8.2 Tools

Category	Tools
SAST	SonarQube, Semgrep, CodeQL, Checkmarx

DAST	OWASP ZAP, Burp Suite
Container scan	Trivy, Gype, Snyk, Clair
IaC scan	Checkov, tfsec, KICS, cfn-lint
Secrets scan	git-secrets, TruffleHog, Gitleaks
Policy as code	OPA, Gatekeeper, Kyverno, Sentinel
WAF / edge	AWS WAF, Cloudflare, ModSecurity
PKI / certs	cert-manager, Let's Encrypt, ACM
SIEM	Splunk, Elastic Security, AWS Security Hub

2.9 Level 8 — Data, Messaging & Databases (Ops view)

2.9.1 Concepts

- Backup & restore strategies (RPO/RTO)
- Replication, failover, DR
- Connection pooling
- Caching layers
- Message queues & event-driven architecture
- Database migrations in CI/CD

2.9.2 Tools

Category	Tools
RDBMS	PostgreSQL, MySQL, MariaDB, SQL Server
NoSQL	MongoDB, DynamoDB, Redis, Cassandra
Messaging	Kafka, RabbitMQ, SQS, SNS, Pub/Sub
Streaming	Kinesis, Flink, Spark Streaming
Backup	Velero (K8s), AWS Backup, pg_dump, Percona XtraBackup
Migrations	Flyway, Liquibase, Alembic

2.10 Level 9 — Platform Engineering & Internal Developer Platforms (adv)

2.10.1 Concepts

- Platform as a product
- Developer experience (DevEx)
- Self-service portals
- Golden paths & paved roads
- Service catalogs
- Backstage software templates

2.10.2 Tools

Category	Tools

IDP	Backstage, Port, Cortex
API gateways	Kong, Apigee, AWS API Gateway, Ambassador
Service discovery	Consul, Eureka, CoreDNS
Feature flags	LaunchDarkly, Unleash, Flagsmith

2.11 Level 10 — Advanced & Specialized (adv)

2.11.1 Concepts

- Site Reliability Engineering (SRE) — toil reduction, automation
- FinOps & cloud cost governance
- Multi-cloud & hybrid cloud
- Edge computing & CDN strategy
- WebAssembly in ops (adv)
- eBPF for observability (adv)
- ML/LLM ops (MLOps) pipelines

2.11.2 Tools

Category	Tools
FinOps	Kubecost, CloudHealth, AWS Cost Explorer, Infracost
CDN	CloudFront, Cloudflare, Fastly, Akamai
DNS	Route 53, Cloudflare DNS, ExternalDNS
Workflow	Airflow, Temporal, Step Functions
MLOps	MLflow, Kubeflow, SageMaker (AWS)
eBPF	Cilium, Pixie, Falco

2.12 Certifications (optional milestones)

Track	Certifications
Linux	LPIC, RHCSA/RHCE
Cloud	AWS SAA, AWS DVA, AWS SOA, AWS DOP, AWS SAP; Azure AZ-104, AZ-400; GCP PCA
Kubernetes	CKA, CKAD, CKS
Terraform	HashiCorp Terraform Associate
Security	CISSP (broad), AWS Security Specialty
SRE	Google SRE book + practice (no single cert)

2.13 Learning project ideas (apply everything)

1. **Static site pipeline** — S3 + CloudFront + GitHub Actions + Terraform

2. **Microservices on EKS** — Helm, Prometheus, Argo CD
3. **Full observability stack** — app + OpenTelemetry + Grafana stack
4. **Secure CI/CD** — SAST/DAST/SCA in pipeline, signed images
5. **Multi-env IaC** — dev/stage/prod with Terragrunt or Terraform workspaces
6. **Incident simulation** — chaos tests + runbooks + PagerDuty
7. **Internal developer portal** — Backstage with one golden-path service

3 AWS Cloud Curriculum: Services & Tools

(Beginner → Advanced)

All major AWS services grouped by domain, with learning priority.

Legend: ★ = core (learn first) · ◆ = intermediate · ◇ = advanced/specialized

3.1 1. Account, Identity & Governance ★

Service / Tool	Purpose
AWS Organizations ★	Multi-account structure, SCPs
IAM ★	Users, groups, roles, policies, MFA
IAM Identity Center (SSO) ★	Workforce federation, permission sets
AWS Control Tower ◆	Landing zone automation
AWS Config ◆	Resource compliance & history
AWS CloudTrail ★	API audit logging
AWS Service Catalog ◇	Governed self-service provisioning
AWS Resource Access Manager (RAM) ◆	Cross-account resource sharing
AWS Account Factory ◆	Automated account vending (Control Tower)
AWS CLI / CloudShell ★	Command-line & browser shell
AWS Management Console ★	Web UI
AWS CloudFormation ★	Infrastructure as Code (native)
AWS CDK ◆	IaC in TypeScript/Python/etc.
Terraform + AWS Provider ★	Industry-standard IaC
AWS Tagging & Resource Groups ★	Cost & ops organization

3.1.1 Concepts

- Shared responsibility model
- Root account hygiene (MFA, no daily use)
- Least privilege & permission boundaries
- SCP vs IAM policy
- Cross-account roles & assume-role
- AWS global vs regional services

3.2 2. Networking & Content Delivery ★

Service	Purpose
VPC ★	Virtual network, subnets, route tables
Internet Gateway / NAT Gateway ★	Public & private egress

VPC Endpoints (Gateway & Interface) ♦	Private AWS API access
Security Groups & NACLs ★	Stateful vs stateless firewalls
Elastic IP ★	Static public IPs
Route 53 ★	DNS, health checks, routing policies
CloudFront ★	CDN, edge caching
ALB / NLB / GWLB ★	Layer 7, 4, and gateway load balancers
VPC Peering / Transit Gateway ♦	Inter-VPC & hybrid connectivity
AWS VPN / Direct Connect ♦	On-prem hybrid networking
PrivateLink ♦	Private service exposure
AWS Global Accelerator ◇	Anycast for global apps
Route 53 Resolver ♦	Hybrid DNS
AWS Network Firewall ◇	Managed perimeter firewall
VPC Lattice ◇	Service-to-service networking

3.2.1 Tools

- **Reachability Analyzer, VPC Flow Logs, Traffic Mirroring**

3.3 3. Compute ★

Service	Purpose
EC2 ★	Virtual machines, AMIs, instance types
Auto Scaling Groups ★	Horizontal scaling
Elastic Beanstalk ♦	PaaS for web apps
Lambda ★	Serverless functions
ECS / Fargate ★	Container orchestration (AWS-native)
EKS ★	Managed Kubernetes
AWS Batch ♦	Batch/HPC jobs
Lightsail ♦	Simplified VPS
App Runner ♦	Container PaaS
EC2 Image Builder ♦	Automated AMI pipelines
AWS Outposts / Wavelength / Local Zones ◇	Edge & hybrid compute
Serverless Application Repository ◇	Share Lambda apps

3.3.1 Tools

- **AWS Systems Manager** (Session Manager, Run Command, Patch Manager)
- **EC2 Instance Connect**

3.4 4. Storage ★

Service	Purpose
S3 ★	Object storage, versioning, lifecycle
EBS ★	Block storage for EC2
EFS ◆	Managed NFS
FSx ◆	Windows/Lustre/ONTAP/OpenZFS file systems
S3 Glacier ◆	Archive tiers
Storage Gateway ◇	Hybrid storage
AWS Backup ◆	Centralized backup
DataSync / Transfer Family ◆	Large data migration

3.4.1 Concepts

- Storage classes (S3 Standard, IA, Glacier)
- Encryption at rest (SSE-S3, SSE-KMS, SSE-C)
- S3 bucket policies vs ACLs
- EBS volume types (gp3, io2, st1)

3.5 5. Databases ★

Service	Purpose
RDS ★	Managed MySQL, PostgreSQL, MariaDB, Oracle, SQL Server
Aurora ★	AWS-native relational, serverless option
DynamoDB ★	Serverless NoSQL key-value/document
ElastiCache ◆	Redis / Memcached
DocumentDB ◆	MongoDB-compatible
Neptune ◇	Graph database
Timestream ◇	Time-series
Keyspaces ◇	Cassandra-compatible
Redshift ◆	Data warehouse
QLDB ◇	Ledger database
MemoryDB for Redis ◆	Durable Redis
RDS Proxy ◆	Connection pooling

3.6 6. Application Integration & Messaging ★

Service	Purpose
SQS ★	Managed message queues
SNS ★	Pub/sub notifications
EventBridge ★	Event bus, scheduling
Step Functions ◆	Workflow orchestration

MQ ♦	Managed ActiveMQ/RabbitMQ
AppSync ♦	GraphQL API
API Gateway ★	REST/WebSocket/HTTP APIs

3.7 7. Developer Tools & CI/CD ★

Service	Purpose
CodeCommit ♦	Git repos (legacy; many use GitHub)
CodeBuild ★	Managed build
CodeDeploy ♦	Deployment automation
CodePipeline ★	CI/CD orchestration
CodeArtifact ♦	Package repository
Cloud9 ♦	Cloud IDE
CDK / SAM / Copilot ♦	IaC & serverless/container tooling
X-Ray ♦	Distributed tracing
CloudWatch Synthetics ♦	Canary monitoring

3.8 8. Observability & Management ★

Service	Purpose
CloudWatch ★	Metrics, alarms, dashboards, logs
CloudWatch Logs Insights ★	Log query
AWS X-Ray ♦	Tracing
Systems Manager ★	Ops hub (Parameter Store, Patch, Session)
CloudTrail ★	Audit
AWS Health Dashboard ★	Service events
Trusted Advisor ♦	Best-practice checks
Personal Health Dashboard ★	Account-specific events
Service Quotas ♦	Limit management
OpsWorks ◇	Legacy Chef/Puppet (avoid for new projects)

3.9 9. Security, Identity & Compliance ★

Service	Purpose
IAM ★	(see section 1)
KMS ★	Encryption key management

Secrets Manager ★	Rotating secrets
Certificate Manager (ACM) ★	TLS certificates
WAF & Shield ◆	Web firewall & DDoS
GuardDuty ★	Threat detection
Inspector ◆	Vulnerability scanning
Macie ◆	S3 data classification
Security Hub ◆	Central security findings
Detective ◇	Security investigation
Firewall Manager ◆	Central WAF/rule management
Artifact ◆	Compliance reports
Audit Manager ◇	Continuous audit evidence
CloudHSM ◇	Dedicated HSM
Cognito ◆	User pools & identity for apps
Verified Permissions ◇	Cedar policy engine
IAM Access Analyzer ◆	External access findings

3.10 10. Migration & Transfer ◆

Service	Purpose
Migration Hub	Migration tracking
Application Migration Service (MGN)	Lift-and-shift
Database Migration Service (DMS)	DB replication
Server Migration Service	(legacy → MGN)
Transfer Family	SFTP/FTPS/FTP to S3
Snow Family	Physical data transfer appliances

3.11 11. Analytics & Big Data ◆

Service	Purpose
Athena ◆	SQL on S3
Glue ◆	ETL & data catalog
EMR ◆	Hadoop/Spark
Kinesis ◆	Streaming (Data Streams, Firehose, Analytics)
QuickSight ◆	BI dashboards
Lake Formation ◆	Data lake governance
MSK ◆	Managed Kafka
OpenSearch Service ◆	Search & analytics (Elasticsearch fork)

Data Pipeline ◇	Legacy orchestration
Clean Rooms ◇	Privacy-preserving analytics

3.12 12. Machine Learning & AI ◇

Service	Purpose
SageMaker ◇	End-to-end ML platform
Bedrock ◇	Foundation models API
Comprehend / Rekognition / Textract / Polly / Transcribe ◇	AI APIs
Forecast / Personalize / Kendra ◇	Specialized ML
CodeWhisperer / Q Developer ◆	AI-assisted development

3.13 13. IoT, Media & Specialized ◇

Service	Purpose
IoT Core / Greengrass	Device connectivity
MediaConvert / Live / Package	Video processing
GameLift	Game server hosting
WorkSpaces / AppStream	Virtual desktops
Connect	Contact center

3.14 14. Cost & Billing ★

Service / Tool	Purpose
Cost Explorer ★	Spend analysis
Budgets ★	Alerts & limits
Cost & Usage Report (CUR) ◆	Detailed billing export
Pricing Calculator ★	Estimate costs
Savings Plans / Reserved Instances ◆	Commitment discounts
Compute Optimizer ◆	Right-sizing recommendations
Billing Conductor ◇	Reseller/MSP billing

3.14.1 Concepts

- Pay-as-you-go vs reserved vs spot
- Tagging strategy for cost allocation
- FinOps culture & unit economics

3.15 15. Well-Architected Framework (conceptual ★)

Six pillars — learn alongside every service:

1. **Operational Excellence** — run & monitor systems
2. **Security** — protect data & assets
3. **Reliability** — recover from failure
4. **Performance Efficiency** — use resources well
5. **Cost Optimization** — avoid unnecessary spend
6. **Sustainability** — minimize environmental impact

Tool: AWS Well-Architected Tool (workload reviews)

3.16 Certification path (recommended order)

Order	Certification	Focus
1	Cloud Practitioner (CLF)	Broad AWS vocabulary
2	Solutions Architect Associate (SAA)	Design & core services
3	Developer Associate (DVA)	CI/CD, serverless, APIs
4	SysOps Administrator (SOA)	Operations & monitoring
5	DevOps Engineer Professional (DOP)	Advanced CI/CD & automation
6	Solutions Architect Professional (SAP)	Complex multi-account design
7	Security Specialty	Deep security services
8	Networking Specialty	VPC, hybrid, TGW

3.17 Hands-on lab progression

1. Create IAM user, MFA, CLI profile
2. VPC with public/private subnets + NAT
3. EC2 + ALB + Auto Scaling
4. S3 static site + CloudFront + Route 53
5. RDS Multi-AZ + Secrets Manager
6. Lambda + API Gateway + DynamoDB
7. ECS Fargate or EKS cluster with CI/CD
8. CloudWatch dashboards + alarms + X-Ray
9. Terraform modules for reusable infra
10. Multi-account landing zone (Organizations + Control Tower)

4 DevOps Handbook — Master Table of Contents

Target length: ~1,200 pages (~480,000 words)
Audience: Beginner → Advanced
Format: Each numbered section ≈ one “page” of ~400 words in the final edition

4.1 How to read this handbook

Work through **Parts I-XII** in order. Labs (marked) add hands-on time beyond page counts. Cross-reference the [AWS Handbook](#) for cloud-specific depth.

4.2 Part I — Introduction & Mindset (Pages 1-60)

Ch	Title	Pages
1	What Is DevOps? History and Culture	1-15
2	CALMS, Three Ways, and Team Topologies	16-30
3	DevOps vs SRE vs Platform Engineering	31-42
4	Career Paths, Certifications, and Learning Strategy	43-60

Written: [Chapter 1](#), [Chapter 2](#)

4.3 Part II — Linux & Shell Mastery (Pages 61-140)

Ch	Title	Pages
5	Linux Fundamentals: Filesystem, Users, Permissions	61-85
6	Processes, systemd, and Service Management	86-105
7	Networking on Linux: ip, ss, curl, dig	106-120
8	Bash Scripting for Automation	121-140

Written: [Chapter 5](#)

4.4 Part III — Git & Collaboration (Pages 141-200)

Ch	Title	Pages
9	Git Internals: Objects, Refs, and the DAG	141-158
10	Branching Strategies and Code Review	159-175
11	GitHub/GitLab Workflows: PRs, Issues, Actions intro	176-200

4.5 Part IV — Networking for DevOps (Pages 201-280)

Ch	Title	Pages
12	OSI Model, TCP/IP, DNS Deep Dive	201-225
13	HTTP/HTTPS, TLS, and Certificates	226-245
14	Load Balancing, Proxies, and CDN Concepts	246-265
15	Cloud Networking Patterns (VPC overview)	266-280

4.6 Part V — Scripting & Programming (Pages 281-360)

Ch	Title	Pages
16	Python for DevOps: boto3, requests, pathlib	281-305
17	Go for CLI Tools and Operators	306-325
18	JSON/YAML, Jinja2, and Templating	326-345
19	Testing Automation Scripts	346-360

4.7 Part VI — Infrastructure as Code (Pages 361-480)

Ch	Title	Pages
20	IaC Principles: State, Idempotency, Drift	361-380
21	Terraform: HCL, Providers, Modules	381-410
22	Terraform: Workspaces, Backends, CI Integration	411-430
23	Ansible: Playbooks, Roles, Inventories	431-450
24	Packer & Golden Images	451-465
25	Policy as Code: OPA, Checkov, Sentinel	466-480

4.8 Part VII — Containers (Pages 481-580)

Ch	Title	Pages
26	Docker: Images, Dockerfile, Multi-stage Builds	481-505
27	Container Networking, Volumes, and Security	506-525
28	Docker Compose and Local Dev Environments	526-540
29	Container Registries and Image Signing	541-560
30	Rootless Containers and Alternatives (Podman)	561-580

4.9 Part VIII — Kubernetes (Pages 581-720)

Ch	Title	Pages
31	K8s Architecture: Control Plane & Node Components	581–605
32	Workloads: Pod, Deployment, StatefulSet, DaemonSet	606–630
33	Services, Ingress, and Network Policies	631–655
34	ConfigMaps, Secrets, and Storage (PV/PVC)	656–675
35	Helm, Kustomize, and GitOps (Argo CD)	676–700
36	RBAC, Pod Security, and Hardening	701–720

4.10 Part IX — CI/CD (Pages 721–840)

Ch	Title	Pages
37	CI/CD Concepts and Pipeline Design	721–745
38	GitHub Actions: Workflows, Secrets, Runners	746–770
39	Jenkins: Pipelines as Code (Declarative)	771–790
40	GitLab CI and Multi-platform Comparison	791–810
41	Deployment Strategies: Blue/Green, Canary, Feature Flags	811–825
42	DORA Metrics and Continuous Improvement	826–840

4.11 Part X — Observability & SRE (Pages 841–960)

Ch	Title	Pages
43	Monitoring Fundamentals: Metrics, Logs, Traces	841–865
44	Prometheus & Grafana Stack	866–890
45	ELK/EFK and Log Aggregation	891–910
46	OpenTelemetry and Distributed Tracing	911–930
47	SLOs, Error Budgets, and Incident Response	931–945
48	Chaos Engineering and Game Days	946–960

4.12 Part XI — Security (DevSecOps) (Pages 961–1080)

Ch	Title	Pages
49	Shift-Left Security in the Pipeline	961–980
50	SAST, DAST, SCA, and Container Scanning	981–1000
51	Secrets Management: Vault, SOPS, External Secrets	1001–1020

52	Supply Chain: SBOM, Sigstore, SLSA	1021-1040
53	Runtime Security: Falco, OPA Gatekeeper	1041-1060
54	Compliance Automation and Audit Trails	1061-1080

4.13 Part XII — Advanced Topics (Pages 1081-1200)

Ch	Title	Pages
55	Platform Engineering & Internal Developer Platforms	1081-1100
56	Multi-Cloud and Hybrid Patterns	1101-1115
57	FinOps and Cost-Aware Engineering	1116-1130
58	MLOps Overview for DevOps Engineers	1131-1145
59	Capstone: End-to-End Production System Design	1146-1180
60	Appendix: Command Cheatsheets, Glossary, Index	1181-1200

4.14 Expansion status

Part	Status
I	Ch 1-2 written (~30 pages)
II	Ch 5 written (~25 pages)
III-XII	Outlined; request by chapter number to generate

To request a chapter: *“Expand DevOps Handbook Chapter 21 (Terraform modules)”*

5 Chapter 1: What Is DevOps? History and Culture

DevOps Handbook — Part I, Pages 1–15

5.1 1.1 The problem DevOps solves

Before DevOps, software organizations often split into two camps:

- **Development** — wrote features quickly, measured by release velocity.
- **Operations** — kept systems stable, measured by uptime and incident count.

These goals frequently conflicted. Developers threw code “over the wall”; operators resisted change because every deployment risked an outage. Releases were rare, painful, and manual. When something broke in production, blame flowed in both directions.

DevOps is not a single tool or job title. It is a **cultural and technical movement** that unifies development and operations around shared goals: deliver value to users **fast** and **reliably**.

5.2 1.2 A working definition

***DevOps** is the practice of aligning people, processes, and technology so that software can be built, tested, released, and operated continuously—with feedback loops that shorten the time from idea to production.*

Key outcomes:

Outcome	What it means
Shorter lead time	Hours or days from commit to production, not months
Higher deployment frequency	Many small releases instead of rare big-bang releases
Lower change failure rate	Most changes succeed without rollback
Faster recovery	When failures happen, restore service quickly

These four metrics are known as **DORA metrics** (DevOps Research and Assessment) and are the standard way to measure DevOps maturity.

5.3 1.3 Brief history

Year	Milestone
2007–2009	Flickr, Etsy, and others share “10+ deploys per day” stories
2009	First DevOpsDays conference in Ghent, Belgium (Patrick Debois)
2010	“The Phoenix Project” novel popularizes DevOps narrative
2013	“The DevOps Handbook” (Gene Kim et al.) codifies practices
2016+	Cloud-native, containers, and Kubernetes accelerate adoption
2020s	Platform engineering, GitOps, and AI-assisted ops emerge

The term **DevOps** blends **Development** + **Operations**. It intentionally breaks down silos rather than creating a third silo called “DevOps team” that sits between dev and ops.

5.4 1.4 Culture before tools

A common mistake is buying Jenkins, Kubernetes, or Terraform and calling it “DevOps.” Tools enable DevOps; they do not create it.

Cultural pillars:

1. **Shared ownership** — The team that builds a service also runs it (“you build it, you run it”).
 2. **Blameless postmortems** — Focus on systems and processes, not individuals, after incidents.
 3. **Automation of toil** — Repetitive manual work is eliminated or codified.
 4. **Continuous learning** — Experimentation, feedback, and improvement are expected.
 5. **Customer focus** — Every decision ties back to user value.
-

5.5 1.5 DevOps practices (overview)

You will study each in depth later in this handbook:

Practice	Summary
Version control	Everything in Git: code, config, IaC, docs
CI/CD	Automated build, test, deploy pipelines
Infrastructure as Code	Reproducible environments (Terraform, Ansible)
Monitoring & observability	Know what production is doing in real time
Microservices & containers	Smaller deployable units (optional but common)
Security integration (DevSecOps)	Security checks in the pipeline, not at the end

5.6 1.6 Organizational models

5.6.1 Functional silos (anti-pattern for DevOps)

Separate dev, QA, ops, and security teams with handoffs.

5.6.2 Cross-functional product teams (recommended)

One team owns a product or service end-to-end: code, infra, on-call, roadmap.

5.6.3 DevOps enablement / platform team

A **platform team** builds internal tools (CI templates, K8s clusters, golden paths) so product teams move faster. This is **not** a ticket-taking ops team—it is a **product for developers**.

5.7 1.7 What DevOps is NOT

- **Not “NoOps”** — Operations expertise is more important at scale, not less.
- **Not only for startups** — Enterprises adopt DevOps for compliance and speed.

- **Not replacing developers with operators** — It merges responsibilities skillfully.
- **Not a certification checkbox** — Culture and practice matter more than badges.

5.8 1.8 Your learning path in this handbook

```
graph LR; F[Foundations] --> A[Automation]; A --> C[Containers]; C --> CI[CI/CD]; CI --> O[Observability]; O --> S[Security]; S --> Adv[Advanced]; L["Linux, Git, Networking"] -.-> P["Platform & SRE"]; L --- F; P --- Adv;
```

Invest 70% of study time in **hands-on labs**. Reading without doing builds false confidence.

5.9 1.9 Chapter summary

- DevOps unifies dev and ops around **speed** and **reliability**.
- **Culture** (shared ownership, blameless learning) precedes tools.
- Measure progress with **DORA metrics**.
- Cross-functional teams and platform engineering are modern expressions of DevOps.

5.10 Lab 1.1 — Reflect on your current workflow

1. Map your (or a sample) team's path from code commit to production.
2. Count manual steps and handoffs.
3. Identify one bottleneck and one automation opportunity.
4. Write a one-paragraph "DevOps vision" for that team.

5.11 Review questions

1. Name the four DORA metrics.
2. Why is “DevOps team” as a silo problematic?
3. What year and event is credited with coining “DevOps”?
4. Give one cultural and one technical practice of DevOps.

Next: Chapter 2 — CALMS, Three Ways, and Team Topologies

6 Chapter 2: CALMS, Three Ways, and Team Topologies

DevOps Handbook — Part I, Pages 16–30

6.1 2.1 The CALMS framework

CALMS is a mnemonic for assessing DevOps maturity across five dimensions:

Letter	Dimension	Description
C	Culture	Collaboration, trust, shared goals
A	Automation	Eliminate manual, error-prone work
L	Lean	Reduce waste, small batches, flow
M	Measurement	Data-driven decisions, DORA metrics
S	Sharing	Knowledge, tools, failures, successes

6.1.1 Culture (C)

Healthy DevOps culture means developers understand production constraints and operators understand business priorities. Psychological safety allows people to report problems early.

6.1.2 Automation (A)

Automate builds, tests, deployments, infrastructure provisioning, and compliance checks. Manual runbooks become code. **Toil**—repetitive operational work that scales linearly with service growth—should be targeted for elimination.

6.1.3 Lean (L)

Inspired by Toyota Production System: optimize **flow**, limit work in progress (WIP), and deliver **small batches**. Large releases hide defects; small releases surface them quickly.

6.1.4 Measurement (M)

You cannot improve what you do not measure. Track deployment frequency, lead time, MTTR, change failure rate, plus business KPIs (revenue, user satisfaction).

6.1.5 Sharing (S)

Internal conferences, shared Slack channels, communal dashboards, and open postmortems spread expertise. Tools and patterns are reused, not reinvented per team.

6.2 2.2 The Three Ways (Gene Kim)

From *The Phoenix Project* and *The DevOps Handbook*, the **Three Ways** describe the principles of DevOps flow:

6.2.1 First Way: Flow (left to right)

Accelerate delivery from **development** → **operations** → **customer**.

- Make work visible (Kanban boards, pipeline status).
- Reduce batch sizes.
- Build quality in (automated tests at every stage).
- Never pass defects downstream.

Example: A commit triggers CI; only green builds reach staging; only approved artifacts reach production.

6.2.2 Second Way: Feedback (right to left)

Amplify feedback from **production** → **development**.

- Monitor user experience and system health.
- Swarm on problems; shorten feedback loops.
- Developers get production access (with guardrails).

Example: Error rate spike in Grafana pages the on-call engineer; linked traces show the bad deploy; rollback is automated.

6.2.3 Third Way: Continuous learning and experimentation

Create a culture of **improvement** and **risk-taking**.

- Blameless postmortems.
- Reserve time for improvement (20% ops time, hack days).
- Rehearse failures (game days, chaos engineering).

6.3 2.3 Team Topologies (Skelton & Pais)

Modern DevOps organizations structure teams intentionally:

Team type	Purpose
Stream-aligned	Delivers product features; primary value stream
Platform	Provides self-service infra (K8s, CI, databases)
Enabling	Coaches stream teams on new tech (temporary)
Complicated-subsystem	Owns deep specialty (e.g., video encoding, ML)

6.3.1 Interaction modes

- **Collaboration** — Two teams work closely for discovery.
- **X-as-a-Service** — Platform provides API/portal; stream teams consume.
- **Facilitating** — Enabling team mentors others.

Anti-pattern: One ops team serves 50 dev teams via tickets.

Better: Platform team offers paved roads; stream teams deploy autonomously.

6.4 2.4 Conway's Law and reverse Conway maneuver

“Organizations design systems that mirror their communication structure.” — Melvin Conway

If dev and ops are separate departments, you will get separate deployment artifacts and fragile handoffs.

Reverse Conway maneuver: Structure teams the way you want the architecture to look. Want autonomous microservices? Give teams end-to-end ownership of each service.

6.5 2.5 Maturity model (informal)

Level	Characteristics
1 — Initial	Manual deploys, silos, hero culture
2 — Managed	Some CI, version control, basic monitoring
3 — Defined	IaC, automated testing, standardized pipelines
4 — Measured	SLOs, DORA tracking, blameless postmortems
5 — Optimizing	Continuous experimentation, platform self-service

Use this for self-assessment, not as a rigid certification ladder.

6.6 2.6 Chapter summary

- **CALMS** evaluates Culture, Automation, Lean, Measurement, Sharing.
 - The **Three Ways** optimize flow, feedback, and learning.
 - **Team Topologies** align org structure with delivery goals.
 - Apply **Conway's Law** consciously when designing teams.
-

6.7 Lab 2.1 — CALMS assessment

Score your team 1–5 on each CALMS letter. Pick the lowest score and draft one actionable improvement per dimension.

6.8 Review questions

1. What does the “L” in CALMS stand for, and what Lean concept supports it?
 2. Describe the First Way in one sentence.
 3. Name the four team types in Team Topologies.
 4. What is a reverse Conway maneuver?
-

Next: Part II — [Chapter 5: Linux Fundamentals](#)

7 Chapter 5: Linux Fundamentals — Filesystem, Users, Permissions

DevOps Handbook — Part II, Pages 61–85

7.1 5.1 Why Linux dominates DevOps

Most servers, containers, and cloud instances run **Linux**. As a DevOps engineer you will SSH into machines, debug production issues, write shell scripts, and configure services daily. Windows matters too (Active Directory, .NET shops), but Linux fluency is non-negotiable.

Common distributions:

Distro	Typical use
Ubuntu	Cloud default, tutorials, desktops
Debian	Stable base for Ubuntu and containers
RHEL / Rocky / Alma	Enterprise, regulated industries
Amazon Linux	AWS-optimized

7.2 5.2 Filesystem hierarchy (FHS)

Path	Purpose
/	Root of entire tree
/home	User home directories (/home/alice)
/root	root user’s home
/etc	Configuration files
/var	Variable data: logs (/var/log), caches
/tmp	Temporary files (often cleared on reboot)
/usr	User programs and libraries
/bin , /sbin	Essential binaries
/opt	Optional third-party software
/proc , /sys	Virtual filesystems for kernel info

Everything is a **file** in Unix—including devices and sockets.

7.3 5.3 Essential navigation commands

```
pwd           # Print working directory
ls -lah       # List files (human-readable, including hidden)
cd /etc/nginx # Change directory
cd ..         # Parent directory
cd ~          # Home directory
```

```
tree -L 2 /etc # Tree view (install: apt install tree)
```

Tab completion saves time: type partial names and press `Tab`.

7.4 5.4 File operations

```
cp source dest # Copy
cp -r dir1 dir2 # Copy directory recursively
mv old new # Move or rename
rm file # Delete (careful!)
rm -rf directory # Delete directory recursively (very careful!)
mkdir -p path/to/dir # Create nested directories
touch file.txt # Create empty file or update timestamp
cat file # Print file contents
less /var/log/syslog # Paginated view (q to quit)
head -n 20 file # First 20 lines
tail -f /var/log/app.log # Follow log in real time
```

7.4.1 Finding files

```
find /var/log -name "*.log" -mtime -7 # Logs modified in 7 days
find / -type f -size +100M 2>/dev/null # Files larger than 100MB
locate nginx.conf # Fast index search (updatedb)
```

7.5 5.5 Users, groups, and permissions

7.5.1 Users and groups

```
id # Current user UID, GID, groups
whoami
cat /etc/passwd # User accounts
cat /etc/group # Groups
sudo command # Run as root (if permitted)
su - username # Switch user
```

7.5.2 Permission model

Each file has three permission sets for **user (u)**, **group (g)**, **others (o)**:

```
-rwxr-xr-- 1 alice developers 4096 Jan 15 10:00 deploy.sh
| | | | |
| | | | | others: read
| | | | | group: read, execute
| | | | | user: read, write, execute
```

Symbol	Meaning
r (4)	Read
w (2)	Write
x (1)	Execute (directories: enter)

```
chmod 755 script.sh # rwxr-xr-x
chmod u+x script.sh # Add execute for user
chown alice:developers file
chgrp developers file
```

7.5.3 Special permissions

- **setuid** — Run as file owner (e.g., `passwd`)
- **setgid** — Run as group; dirs inherit group
- **sticky bit** — On `/tmp` , only owner can delete own files

7.6 5.6 Package management (Debian/Ubuntu)

```
sudo apt update           # Refresh package index
sudo apt upgrade -y       # Upgrade installed packages
sudo apt install nginx -y # Install package
apt search nginx          # Search
apt show nginx            # Package info
sudo apt remove nginx     # Remove
dpkg -l | grep nginx     # List installed
```

RHEL/Rocky equivalent: `dnf install` , `dnf update` .

7.7 5.7 Text processing (daily DevOps tools)

```
grep -r "ERROR" /var/log/app/ # Search recursively
grep -E "timeout|refused" log.txt # Extended regex
awk '{print $1}' access.log    # Column extraction
sed 's/old/new/g' file.txt    # Stream editor
sort file | uniq -c | sort -rn | head # Count unique lines
wc -l file                    # Line count
```

Pipes (|) chain commands: stdout of one → stdin of next.

7.8 5.8 Environment and shell

```
echo $PATH                # Executable search path
export MY_VAR=hello       # Set environment variable
env                        # All environment variables
which python3             # Path to executable
alias ll='ls -lah'        # Command shortcut
```

Shell config files: `~/.bashrc` , `~/.profile` (sourced on login).

7.9 5.9 SSH — remote access

```
ssh user@hostname
ssh -i ~/.ssh/mykey.pem ec2-user@1.2.3.4
scp file user@host:/remote/path
rsync -avz ./local/ user@host:/remote/ # Efficient sync
```

`~/.ssh/config` simplifies connections:

```
Host prod-web
  HostName 10.0.1.50
  User ubuntu
  IdentityFile ~/.ssh/prod.pem
```

Then: `ssh prod-web`

7.10 5.10 Chapter summary

- Linux FHS organizes `/etc`, `/var`, `/home`, and more.
 - Master `ls`, `cd`, `cp`, `mv`, `rm`, `find`, `grep`, `tail -f`.
 - Permissions: `user/group/other` + `chmod` / `chown`.
 - Use `apt` / `dnf` for packages; SSH for remote ops.
-

7.11 Lab 5.1 — Linux sandbox

1. Create user `devops-lab` with home directory.
 2. Create `~/scripts/deploy.sh` with `#!/bin/bash` and `echo "Deploying..."`.
 3. Set permissions to `755`; run it.
 4. Write a one-liner to count ERROR lines in a sample log file.
 5. Configure SSH key auth to a local VM or cloud instance.
-

7.12 Review questions

1. What directory holds system configuration files?
 2. What does permission `644` mean in symbolic notation?
 3. How do you follow a log file in real time?
 4. What is the difference between `cp` and `mv`?
-

Continue: Chapter 6 — Processes, systemd, and Service Management (outline in TOC)

8 AWS Handbook — Master Table of Contents

Target length: ~800 pages (~320,000 words)
Audience: Beginner → Advanced AWS practitioners
Companion: [DevOps Handbook](#)

8.1 Part I — Cloud Foundations (Pages 1-80)

Ch	Title	Pages
1	Introduction to AWS & Shared Responsibility	1-18
2	AWS Global Infrastructure: Regions, AZs, Edge	19-35
3	AWS Account Setup, Billing, and Cost Basics	36-55
4	AWS CLI, CloudShell, and SDK Overview	56-80

Written: [Chapter 1](#), [Chapter 2](#)

8.2 Part II — Identity & Access Management (Pages 81-160)

Ch	Title	Pages
5	IAM Users, Groups, Roles, and Policies	81-105
6	IAM Best Practices & Permission Boundaries	106-125
7	IAM Identity Center (SSO) & Federation	126-145
8	AWS Organizations & Service Control Policies	146-160

Written: [Chapter 5](#)

8.3 Part III — Networking (Pages 161-280)

Ch	Title	Pages
9	VPC Fundamentals: CIDR, Subnets, Route Tables	161-185
10	Internet Gateway, NAT, and Routing	186-205
11	Security Groups vs NACLs	206-220
12	Load Balancers: ALB, NLB, GWLB	221-245
13	Route 53: DNS Records and Routing Policies	246-265
14	VPC Peering, Transit Gateway, PrivateLink	266-280

8.4 Part IV — Compute (Pages 281-400)

Ch	Title	Pages
----	-------	-------

15	EC2: Instance Types, AMIs, Launch Templates	281–305
16	Auto Scaling & Elastic Load Balancing Integration	306–325
17	Lambda: Events, Concurrency, and Layers	326–350
18	ECS & Fargate	351–375
19	EKS: Managed Kubernetes on AWS	376–400

8.5 Part V — Storage (Pages 401–480)

Ch	Title	Pages
20	S3: Buckets, Objects, Versioning, Lifecycle	401–425
21	S3 Security, Encryption, and Static Websites	426–445
22	EBS & EFS	446–465
23	AWS Backup & Disaster Recovery Patterns	466–480

8.6 Part VI — Databases (Pages 481–560)

Ch	Title	Pages
24	RDS & Aurora	481–505
25	DynamoDB: Tables, Indexes, Streams	506–525
26	ElastiCache & Caching Patterns	526–540
27	Database Migration (DMS) & DR	541–560

8.7 Part VII — Application Integration (Pages 561–620)

Ch	Title	Pages
28	API Gateway	561–580
29	SQS, SNS, and EventBridge	581–605
30	Step Functions Workflows	606–620

8.8 Part VIII — DevOps on AWS (Pages 621–700)

Ch	Title	Pages
31	CloudFormation Deep Dive	621–645
32	AWS CDK	646–665
33	CodePipeline, CodeBuild, CodeDeploy	666–685
34	Systems Manager & Parameter Store	686–700

8.9 Part IX — Observability & Operations (Pages 701-760)

Ch	Title	Pages
35	CloudWatch Metrics, Alarms, Dashboards	701-720
36	CloudWatch Logs & Insights	721-735
37	X-Ray & Distributed Tracing	736-750
38	CloudTrail & AWS Config	751-760

8.10 Part X — Security & Compliance (Pages 761-800)

Ch	Title	Pages
39	KMS, Secrets Manager, ACM	761-775
40	GuardDuty, Security Hub, WAF, Shield	776-790
41	Well-Architected Review & Capstone Design	791-800

8.11 Expansion status

Part	Status
I	Ch 1-2 written (~35 pages)
II	Ch 5 written (~25 pages)
III-X	Outlined; request by chapter number

To request a chapter: *“Expand AWS Handbook Chapter 15 (EC2 deep dive)”*

9 Chapter 1: Introduction to AWS & the Shared Responsibility Model

AWS Handbook — Part I, Pages 1–18

9.1 1.1 What is Amazon Web Services (AWS)?

Amazon Web Services (AWS) is a comprehensive **cloud computing platform** offering on-demand compute, storage, databases, networking, machine learning, and 200+ other services. Instead of buying physical servers, you provision resources in minutes and pay primarily for what you use.

9.1.1 Why AWS matters for DevOps

- **API-driven** — Every service is controllable via CLI, SDK, and IaC.
 - **Global scale** — Deploy in multiple regions for resilience and latency.
 - **Managed services** — Offload undifferentiated heavy lifting (RDS, EKS, Lambda).
 - **Ecosystem** — Largest cloud market share; extensive documentation and community.
-

9.2 1.2 Cloud service models

Model	You manage	Provider manages	AWS examples
IaaS	OS, apps, data	Hardware, network, hypervisor	EC2, VPC, EBS
PaaS	Apps, data	Runtime, OS, infra	Elastic Beanstalk, App Runner
SaaS	Configuration, data	Everything else	WorkSpaces, Chime
FaaS	Function code	Everything else	Lambda

Containers blur lines: ECS/EKS give you orchestration (between IaaS and PaaS).

9.3 1.3 The Shared Responsibility Model

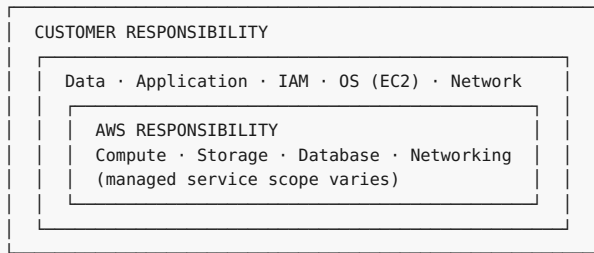
Security and compliance are **shared** between AWS and the customer. The boundary shifts by service type.

9.3.1 AWS is responsible for Security OF the Cloud

- Physical data centers
- Hardware and global network
- Virtualization layer
- Managed service patching (for fully managed services)

9.3.2 You are responsible for Security IN the Cloud

- Data classification and encryption choices
- IAM users, roles, and policies
- Operating system patches (on EC2)
- Network configuration (security groups, NACLs)
- Application code security



9.3.3 Example: RDS vs EC2 MySQL

Task	EC2 + self-managed MySQL	Amazon RDS
Patch MySQL	You	AWS
Patch OS	You	AWS
Configure security groups	You	You
Encrypt data at rest	You (configure)	You (enable)
Physical security	AWS	AWS

9.4 1.4 Core AWS concepts

Concept	Definition
Region	Geographic area (e.g., <code>us-east-1</code> , <code>eu-west-1</code>)
Availability Zone (AZ)	Isolated data center within a region
Account	Billing and security boundary (12-digit ID)
Resource	Any AWS entity (EC2 instance, S3 bucket, etc.)
ARN	Amazon Resource Name — unique identifier
Tag	Key-value metadata on resources

Example ARN:

```
arn:aws:s3:::my-bucket
arn:aws:ec2:us-east-1:123456789012:instance/i-0abcd1234
```

9.5 1.5 AWS console, CLI, and IaC

Three ways to interact with AWS:

1. **Management Console** — Web UI; great for learning and ad-hoc tasks.
2. **AWS CLI** — Scriptable; essential for automation.
3. **Infrastructure as Code** — CloudFormation, CDK, Terraform for reproducible environments.

DevOps engineers spend most time in **CLI + IaC**, using the console for debugging.

9.6 1.6 Pricing fundamentals

- **On-Demand** — Pay per hour/second; no commitment.
- **Reserved Instances / Savings Plans** — 1–3 year commitment; lower cost.
- **Spot Instances** — Unused capacity; cheap but can be interrupted.
- **Free Tier** — Limited free usage for 12 months (and some always-free services).

Always set billing alarms (AWS Budgets) on new accounts.

9.7 1.7 Well-Architected Framework (preview)

AWS defines six pillars for good architecture (covered throughout this handbook):

1. Operational Excellence
 2. Security
 3. Reliability
 4. Performance Efficiency
 5. Cost Optimization
 6. Sustainability
-

9.8 1.8 Chapter summary

- AWS provides on-demand cloud infrastructure and managed services.
 - **Shared responsibility** splits security between AWS and you—know your side.
 - Learn **regions, accounts, ARNs, and tags** early.
 - Use CLI and IaC for production; console for exploration.
-

9.9 Lab 1.1 — Create your AWS account

1. Sign up at <https://aws.amazon.com> (use a dedicated email).
 2. Enable **MFA** on the root account; do not use root daily.
 3. Create an **IAM admin user** for daily work.
 4. Set a **billing budget** alert at \$10 or your comfort threshold.
 5. Explore the console: EC2, S3, IAM dashboards.
-

9.10 Review questions

1. What is the difference between IaaS and PaaS? Give one AWS example of each.
 2. Who patches the OS on an EC2 instance?
 3. What is an ARN used for?
 4. Name two AWS pricing models.
-

Next: [Chapter 2 — Global Infrastructure](#)

10 Chapter 2: AWS Global Infrastructure — Regions, AZs, and Edge

AWS Handbook — Part I, Pages 19–35

10.1 2.1 Global infrastructure overview

AWS operates a worldwide network of data centers organized into:

Region (e.g., us-east-1)

- Availability Zone A (us-east-1a)
- Availability Zone B (us-east-1b)
- Availability Zone C (us-east-1c)
- ... (typically 3+ AZs per region)

Edge Locations (200+) — CloudFront, Route 53, etc.

10.2 2.2 Regions

A **Region** is a separate geographic area. Each region is **fully isolated** from others for most services.

10.2.1 Choosing a region

Factor	Guidance
Latency	Place resources near users
Compliance	Data residency (GDPR, etc.) may require EU regions
Service availability	New services launch in select regions first
Cost	Pricing varies by region (us-east-1 often cheapest)
Disaster recovery	Use a second region for critical workloads

Naming: us-east-1 (N. Virginia), eu-west-1 (Ireland), ap-southeast-1 (Singapore).

10.2.2 Regional vs global services

Regional (must specify region)	Global (single endpoint)
EC2, RDS, VPC, Lambda	IAM, Route 53, CloudFront
S3 (bucket tied to region)	AWS Organizations

10.3 2.3 Availability Zones (AZs)

An **AZ** is one or more discrete data centers with independent power, networking, and cooling within a region. AZs are connected by **low-latency private fiber**.

- Deploy across **multiple AZs** for high availability.
- AZ names like us-east-1a are **account-specific mappings**— 1a in your account may not be the same physical DC as 1a in another account.

10.3.1 Multi-AZ patterns

Pattern	Use case
Active-passive	RDS Multi-AZ failover
Active-active	ALB spanning AZs with EC2 in each
Stretch cluster	EKS node groups per AZ

10.4 2.4 Local Zones, Wavelength, and Outposts

Type	Purpose
Local Zones	Extend region closer to metro areas (ultra-low latency)
Wavelength	5G edge at carrier locations
Outposts	AWS hardware on-premises; hybrid cloud

These are specialized; start with standard regions + AZs.

10.5 2.5 Edge locations and CloudFront

Edge locations cache content close to users via **Amazon CloudFront** (CDN). Route 53 and AWS Shield also use the edge network.

Benefits: - Lower latency for static assets - Reduced origin load - DDoS protection at edge (with Shield)

10.6 2.6 Fault isolation design principles

1. **Never rely on a single AZ** for production.
2. **Design for regional failure** for mission-critical systems (multi-region active-active or pilot light).
3. **Use managed services** that handle AZ redundancy (RDS Multi-AZ, S3 cross-AZ durability).
4. **Test failover** regularly.

10.6.1 S3 durability vs availability

- **S3 Standard** — 99.999999999% (11 nines) durability across AZs in a region.
- **Availability** — Different metric; rare regional outages can still occur.

10.7 2.7 Service quotas (limits)

Each account has **default quotas** per region (e.g., EC2 vCPUs, VPCs per region). Request increases via **Service Quotas** console. Plan capacity before large deployments.

10.8 2.8 Chapter summary

- **Regions** are geographic; **AZs** are isolated DCs within a region.

- Deploy production workloads across **multiple AZs**.
 - **Edge locations** power CloudFront and global services.
 - Know **regional vs global** services when designing architecture.
-

10.9 Lab 2.1 — Explore regions in CLI

```
aws configure                                # Set default region
aws ec2 describe-availability-zones --region us-east-1
aws ec2 describe-regions --output table
```

Document which region you will use for labs and why.

10.10 Review questions

1. How many AZs should a highly available web app use at minimum?
 2. Is IAM a regional or global service?
 3. What AWS service uses edge locations for caching?
 4. Why might you choose `eu-central-1` over `us-east-1`?
-

Next: Part II — [Chapter 5: IAM Fundamentals](#)

11 Chapter 5: IAM Users, Groups, Roles, and Policies

AWS Handbook — Part II, Pages 81–105

11.1 5.1 What is IAM?

AWS Identity and Access Management (IAM) controls **who** can access **which** AWS resources and **how**. Every API call to AWS is authenticated and authorized through IAM (except public resources like anonymous S3 access).

IAM is **global**—users and roles exist at the account level, not per region.

11.2 5.2 IAM identity types

Identity	Use case
Root user	Account owner; created at signup. Avoid for daily use.
IAM user	Long-term credentials for humans or legacy apps
IAM role	Temporary credentials; assumed by users, services, or other accounts
IAM group	Collection of users with shared permissions

Best practice: Humans use **IAM Identity Center (SSO)** or roles with MFA. Applications and EC2/Lambda use **roles**, not access keys on users.

11.3 5.3 Policies — the core of permissions

A **policy** is a JSON document defining **Allow** or **Deny** for **actions** on **resources** under **conditions**.

11.3.1 Policy structure

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3Read",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3::my-app-bucket",
        "arn:aws:s3::my-app-bucket/*"
      ],
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "203.0.113.0/24"
        }
      }
    }
  ]
}
```

```
}
```

11.3.2 Policy types

Type	Attached to
Identity-based	Users, groups, roles
Resource-based	S3 buckets, SNS topics, etc.
Permission boundaries	Max permissions for identity
SCP	AWS Organizations — account ceiling
Session policies	Limit role session scope

Evaluation logic: Explicit **Deny** always wins. Default is implicit deny until Allow.

11.4 5.4 Users and groups

11.4.1 Create a developer group (example workflow)

1. Create group `Developers`.
2. Attach policy `PowerUserAccess` (or custom least-privilege policy).
3. Create users; add to group.
4. Enable **MFA** for console access.
5. Provide **programmatic access** only if needed (prefer roles).

```
aws iam create-group --group-name Developers
aws iam attach-group-policy --group-name Developers \
  --policy-arn arn:aws:iam::aws:policy/PowerUserAccess
aws iam create-user --user-name alice
aws iam add-user-to-group --user-name alice --group-name Developers
```

11.5 5.5 Roles and trust policies

A **role** has two policies:

1. **Trust policy** — Who can assume the role
2. **Permission policy** — What the role can do

11.5.1 EC2 instance role example

Trust policy (allows EC2 service to assume):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Attach permission policy allowing `s3:GetObject` on a specific bucket. The EC2 instance receives **temporary credentials** via instance metadata—no hardcoded keys.

11.5.2 Assume role (cross-account or CLI)

```
aws sts assume-role \
  --role-arn arn:aws:iam::123456789012:role/DeployRole \
  --role-session-name deploy-session
```

11.6 5.6 AWS managed vs customer managed policies

Type	Description
AWS managed	Maintained by AWS (<code>ReadOnlyAccess</code> , <code>AdministratorAccess</code>)
Customer managed	You create and reuse across identities
Inline	Embedded in single user/group/role; harder to reuse

Prefer **customer managed** policies for team standards; avoid `AdministratorAccess` except break-glass accounts.

11.7 5.7 Least privilege in practice

1. Start with **read-only** access; add permissions as needed.
2. Use **IAM Access Analyzer** to find overly broad access.
3. Scope resources with ARNs, not `"Resource": "*" .`
4. Use **conditions** (MFA present, source IP, tag-based).
5. Rotate or eliminate **access keys**; prefer roles.

11.7.1 Tag-based access (ABAC)

```
"Condition": {
  "StringEquals": {
    "aws:PrincipalTag/Department": "${aws:ResourceTag/Department}"
  }
}
```

Users tagged `Department=Engineering` access only resources tagged the same.

11.8 5.8 Common IAM actions for DevOps

Action prefix	Service
<code>ec2:*</code>	EC2
<code>s3:*</code>	S3
<code>iam:PassRole</code>	Allow attaching roles to services
<code>sts:AssumeRole</code>	Switch roles
<code>cloudformation:*</code>	Stack operations

`iam:PassRole` is often required when CI/CD deploys resources that need an instance/task role.

11.9 5.9 IAM Access Analyzer & CloudTrail

- **Access Analyzer** — Identifies resources shared externally.
- **CloudTrail** — Logs every IAM API call for audit.

Enable CloudTrail in all regions for security accounts.

11.10 5.10 Chapter summary

- IAM controls authentication and authorization globally.
 - Use **groups** for humans, **roles** for services and temporary access.
 - Policies are JSON with Effect, Action, Resource, Condition.
 - Apply **least privilege**; avoid root and long-lived access keys.
-

11.11 Lab 5.1 — IAM hands-on

1. Create IAM user `lab-user` with MFA.
 2. Create customer managed policy allowing `s3:ListBucket` and `s3:GetObject` on one bucket.
 3. Create role `EC2-S3-ReadRole` with trust for EC2 and the S3 read policy.
 4. Launch EC2 with the role; verify `aws s3 ls` works without configured keys.
 5. Review effective permissions in **IAM Policy Simulator**.
-

11.12 Review questions

1. What is the difference between an IAM user and an IAM role?
 2. What happens when Allow and Deny conflict?
 3. Why should EC2 instances use roles instead of access keys?
 4. What does `sts:AssumeRole` do?
-

Continue: Chapter 6 — IAM Best Practices (outline in TOC)